

# Data Science, VL 20

Dr. Konrad Völkel, Sommersemester 2026

30.06.2026

Heinrich Heine
Universität Düsseldorf

# Lineare Regression

# Beispiel zur Linearen Regression

Wir haben bereits ein Beispiel gesehen: Blütenblattbreite aus Blütenblattlänge.

# Beispiel zur Linearen Regression

Wir haben bereits ein Beispiel gesehen: Blütenblattbreite aus Blütenblattlänge.

Weiteres Beispiel: Mit einem Datensatz von Häusern in einer australischen Großstadt wollen wir den Hauspreis Y vorhersagen aus der Entfernung zum Stadtzentrum $X_1$ und dem Alter des Hauses $X_2$.

# Beispiel zur Linearen Regression

Wir haben bereits ein Beispiel gesehen: Blütenblattbreite aus Blütenblattlänge.

Weiteres Beispiel: Mit einem Datensatz von Häusern in einer australischen Großstadt wollen wir den Hauspreis Y vorhersagen aus der Entfernung zum Stadtzentrum $X_1$ und dem Alter des Hauses $X_2$.
Das lineare Modell ist $Y = \beta_1 X_1 + \beta_2 X_2 + \beta_0 + \epsilon$.

# Beispiel zur Linearen Regression

Heinrich Heine Universität Düsseldorf

Wir haben bereits ein Beispiel gesehen: Blütenblattbreite aus Blütenblattlänge.

Weiteres Beispiel: Mit einem Datensatz von Häusern in einer australischen Großstadt wollen wir den Hauspreis Y vorhersagen aus der Entfernung zum Stadtzentrum $X_1$ und dem Alter des Hauses $X_2$.
Das lineare Modell ist $Y = \beta_1 X_1 + \beta_2 X_2 + \beta_0 + \epsilon$.
Achtung: In der Informatik / im maschinellen Lernen üblich:
$f(x_1, x_2; \beta_0, \beta_1, \beta_2) := \beta_1 x_2 + \beta_2 x_2 + \beta_0$ wird linear genannt, obwohl nur
$(x_1, x_2) \mapsto \beta_1 x_1 + \beta_2 x_2$ wirklich linear ist;
f heißt in der Mathematik affin-lineare Funktion.

# Beispiel zur Linearen Regression

Heinrich Heine
Universität Düsseldorf

Wir haben bereits ein Beispiel gesehen: Blütenblattbreite aus Blütenblattlänge.

Weiteres Beispiel: Mit einem Datensatz von Häusern in einer australischen Großstadt wollen wir den Hauspreis Y vorhersagen aus der Entfernung zum Stadtzentrum $X_1$ und dem Alter des Hauses $X_2$.
Das lineare Modell ist $Y = \beta_1 X_1 + \beta_2 X_2 + \beta_0 + \epsilon$.
Achtung: In der Informatik / im maschinellen Lernen üblich:
$f(x_1, x_2; \beta_0, \beta_1, \beta_2) := \beta_1 x_2 + \beta_2 x_2 + \beta_0$ wird linear genannt, obwohl nur $(x_1, x_2) \mapsto \beta_1 x_1 + \beta_2 x_2$ wirklich linear ist;
f heißt in der Mathematik affin-lineare Funktion.

Wie finden wir nun 'gute' Parameter $\beta$ und was ist 'gut' für uns?

# Lineare Regression, formal

Ein Regressionsmodell der Form $Y = f(X; w) + \epsilon$ mit $f(x; w) = w^T x$ heißt lineare Regression.

# Lineare Regression, formal

Ein Regressionsmodell der Form $Y = f(X; w) + \epsilon$ mit $f(x; w) = w^T x$ heißt lineare Regression. Die Parameter w heißen auch Gewichte (weights).

# Lineare Regression, formal

Ein Regressionsmodell der Form $Y = f(X; w) + \epsilon$ mit $f(x; w) = w^T x$ heißt lineare Regression. Die Parameter $w$ heißen auch Gewichte (weights).

Ein Regressionsmodell der Form $Y = f(X; w, w_0) + \epsilon$ mit $f(x; w) = w^T x + w_0$ heißt lineare Regression mit Bias $w_0$.

# Lineare Regression, formal

Ein Regressionsmodell der Form $Y = f(X; w) + \epsilon$ mit $f(x; w) = w^T x$ heißt lineare Regression. Die Parameter $w$ heißen auch Gewichte (weights).

Ein Regressionsmodell der Form $Y = f(X; w, w_0) + \epsilon$ mit $f(x; w) = w^T x + w_0$ heißt lineare Regression mit Bias $w_0$.

Wenn wir statt $(X_1, \ldots, X_k)$ als Eingabe $X_+ := (1, X_1, \ldots, X_k)$ verwenden und nicht $w = (w_1, \ldots, w_k)$ sondern $w_+ := (w_0, w_1, \ldots, w_k)$ dann ist $(w_+)^T X_+ = w^T X + w_0$.

Daher ist jede lineare Regression mit Bias auch eine sehr spezielle lineare Regression ohne Bias.

# Lineare Regression mit Scipy.optimize

```python
from scipy.optimize import curve_fit

def f(x, beta_0, beta_1):
    return beta_1*x + beta_0

params, covariance_matrix = curve_fit(f, x, y)
```

# Lineare Regression mit Scipy.optimize

```python
from scipy.optimize import curve_fit

def f(x, beta_0, beta_1):
    return beta_1*x + beta_0

params, covariance_matrix = curve_fit(f, x, y)
```

Die Variable `params` ist das 2-Tupel $\beta$.

Aus der Dokumentation von `curve_fit`:
Use non-linear least squares to fit a function, f, to data.
Assumes `ydata = f(xdata, *params) + eps`.

# Lineare Regression als Schätzer

Heinrich Heine
Universität Düsseldorf

**Definition**

Wenn man in einem Regressionsmodell die Residuenquadratsumme $\sum_{i=1}^{N} (y_i - f(x_i; \beta))^2$ minimiert, spricht man von einem Schätzer für $\beta$, der als kleinste-Quadrate-Schätzer (least squares) bekannt ist.

# Lineare Regression als Schätzer

Heinrich Heine
Universität Düsseldorf

**Definition**

Wenn man in einem Regressionsmodell die Residuenquadratsumme $\sum_{i=1}^{N} (y_i - f(x_i; \beta))^2$ minimiert, spricht man von einem Schätzer für $\beta$, der als kleinste-Quadrate-Schätzer (least squares) bekannt ist.

Der kleinste-Quadrate-Schätzer ist ein Maximum-Likelihood-Schätzer.

# Kleinste Quadrate

Heinrich Heine
Universität Düsseldorf

Wenn wir zu Punkten $(x_i, y_i)$ in der Ebene eine Gerade berechnen, die den gemeinsamen Abstand zu allen Punkten minimiert (eine sogenannte Ausgleichsgerade), so ist dies genau eine Regression mit $f(x, \beta) = \beta_0 + \beta_1 x$. Die Regressionsgleichung $Y_i = f(X_i, \beta)$ wird zur Geradengleichung $y_i = \beta_0 + \beta_1 x_i$ wobei $\beta_0$ der y-Achsenabschnitt (bei $x_i = 0$) und $\beta_1$ die Steigung der Geraden ist.

# Kleinste Quadrate

Heinrich Heine
Universität Düsseldorf

Wenn wir zu Punkten $(x_i, y_i)$ in der Ebene eine Gerade berechnen, die den gemeinsamen Abstand zu allen Punkten minimiert (eine sogenannte Ausgleichsgerade), so ist dies genau eine Regression mit $f(x, \beta) = \beta_0 + \beta_1 x$. Die Regressionsgleichung $Y_i = f(X_i, \beta)$ wird zur Geradengleichung $y_i = \beta_0 + \beta_1 x_i$ wobei $\beta_0$ der y-Achsenabschnitt (bei $x_i = 0$) und $\beta_1$ die Steigung der Geraden ist.

Die kleinste-Quadrate-Methode ist nun, $\beta_0$ und $\beta_1$ so zu schätzen, dass $\sum_{i=1}^{N} (y_i - \beta_1 x_i - \beta_0)^2$ minimiert wird.

# Kleinste Quadrate von Hand, pt. 1

Heinrich Heine
Universität Düsseldorf

Wenn $\beta_0 = 0$ ist (die Ausgleichsgerade durch den Ursprung verläuft), ist $\beta_1$ leicht zu schätzen: Wir leiten die Residuenquadratsumme nach $\beta_1$ ab und finden Nullstellen.

# Kleinste Quadrate von Hand, pt. 1

Heinrich Heine Universität Düsseldorf

Wenn $\beta_0 = 0$ ist (die Ausgleichsgerade durch den Ursprung verläuft), ist $\beta_1$ leicht zu schätzen: Wir leiten die Residuenquadratsumme nach $\beta_1$ ab und finden Nullstellen.

$$\begin{aligned} 0 &= \sum_{i=1}^{N} 2(y_i - \beta_1 x_i)(-x_i) \\ &= \sum_{i=1}^{N} 2\left(\beta_1 x_i^2 - y_i x_i\right) \\ &= 2\left(\beta_1 \sum_{i=1}^{N} x_i^2 - \sum_{i=1}^{N} x_i y_i\right) \end{aligned}$$

also ist $\beta_1 = \dfrac{\sum_{i=1}^{N} x_i y_i}{\sum_{i=1}^{N} x_i^2}$.

# Kleinste Quadrate von Hand, pt. 2

Im Allgemeinen lässt sich $\beta_0$ berechnen mit der Formel

$$\beta_0 = \texttt{mean}(\mathrm{y}) - \beta_1 \texttt{mean}(\mathrm{x}).$$

# Kleinste Quadrate von Hand, pt. 2

Im Allgemeinen lässt sich $\beta_0$ berechnen mit der Formel

$$\beta_0 = \mathsf{mean}(\mathrm{y}) - \beta_1 \mathsf{mean}(\mathrm{x}).$$

Um also zunächst $\beta_1$ im Allgemeinen zu bestimmen, verschiebt man die Daten um die Mittelwerte, also ersetzt $\mathrm{x_i}$ durch $\mathrm{x_i'} = \mathrm{x_i} - \mathsf{mean}(\mathrm{x})$ und $\mathrm{y_i}$ durch $\mathrm{y_i'} = \mathrm{y_i} - \mathsf{mean}(\mathrm{y})$. Damit erhält man die Formel

$$\beta_1 = \frac{\sum_{\mathrm{i}=1}^{\mathrm{N}} (\mathrm{x_i} - \mathsf{mean}(\mathrm{x}))(\mathrm{y_i} - \mathsf{mean}(\mathrm{y}))}{\sum_{\mathrm{i}=1}^{\mathrm{N}} (\mathrm{x_i} - \mathsf{mean}(\mathrm{x}))^2}$$

# Lineare Regression mit Scipy.Stats

Heinrich Heine Universität Düsseldorf

Statt curve fitting lässt sich auch direkt lineare Regression verwenden:

```
1 from scipy.stats import linregress
2
3 linregress_result = linregress(x, y)
4 slope, intercept, r, p, se = linregress_result
```

# Lineare Regression mit Scipy.Stats

Statt curve fitting lässt sich auch direkt lineare Regression verwenden:

```
1 from scipy.stats import linregress
2
3 linregress_result = linregress(x, y)
4 slope, intercept, r, p, se = linregress_result
```

Dabei ist `slope` die Steigung, also $\beta_1$ und `intercept` der y-intercept, der y-Achsenabschnitt der Geraden, also der Bias $\beta_0$. Weiterhin sind `r` der Korrelationskoeffizient, `p` ein p-Wert und `se` der quadrierte Fehler (squared error), also das mittlere Residuenquadrat.

# Achtung: Verwechslungsgefahr

Die Regressionsgerade (=Ausgleichsgerade) ist nicht die Richtung maximaler Varianz!

# Achtung: Verwechslungsgefahr

Die Regressionsgerade (=Ausgleichsgerade) ist nicht die Richtung maximaler Varianz!
Zeichnet man die Abstände ein, die in beiden Fällen gemessen werden, sieht man den Unterschied:
Bei der Regressionsgerade minimiert man die (Quadrate der) Residuen, die parallel zur y-Achse verlaufen, bei der Richtung maximaler Varianz minimiert man die Abstände zur Geraden, die senkrecht zur Geraden verlaufen.
(Wir betrachten später noch mehr Bilder im Notebook).

# Multilineare Regression

Heinrich Heine
Universität Düsseldorf

Wenn wir nur einen univariaten Regressor $X = (X_1)$ haben, also Gewicht $w = w_1$ und Bias $w_0$, sprechen wir von einer einfachen linearen Regression.

# Multilineare Regression

Wenn wir nur einen univariaten Regressor $X = (X_1)$ haben, also Gewicht $w = w_1$ und Bias $w_0$, sprechen wir von einer einfachen linearen Regression.
Wenn der Regressor multivariat ist, also $X = (X_1, \ldots, X_k)$ und damit das Gewicht ein Gewichtsvektor $w = (w_1, \ldots, w_k)$ (mit Bias $w_0$) ist, sprechen wir von einer multilinearen Regression.

# Multilineare Regression

Wenn wir nur einen univariaten Regressor $X = (X_1)$ haben, also Gewicht $w = w_1$ und Bias $w_0$, sprechen wir von einer einfachen linearen Regression.
Wenn der Regressor multivariat ist, also $X = (X_1, \ldots, X_k)$ und damit das Gewicht ein Gewichtsvektor $w = (w_1, \ldots, w_k)$ (mit Bias $w_0$) ist, sprechen wir von einer multilinearen Regression.

Während `scipy.stats.linregress` nur einfache lineare Regression unterstützt, bietet Scikit-Learn auch multilineare Regression an mit der Klasse `sklearn.linear_model.LinearRegression`.

# Lineare Regression mit Basisfunktionserweiterung

# Beispiel zur Basisfunktionserweiterung

Gegeben bivariate Daten $(X_1, X_2)$ wie z.B. Blütenblattlänge und -breite, könnten wir auch einen Zusammenhang zwischen Kelchblattlänge (sepal length) Y und der Blütenblattfläche (petal leaf area) vermuten.

# Beispiel zur Basisfunktionserweiterung

Heinrich Heine
Universität Düsseldorf

Gegeben bivariate Daten $(X_1, X_2)$ wie z.B. Blütenblattlänge und -breite, könnten wir auch einen Zusammenhang zwischen Kelchblattlänge (sepal length) Y und der Blütenblattfläche (petal leaf area) vermuten.
Wir bilden ein neues Feature $X_3 := X_1 X_2$ (das Produkt) und betrachten die lineare Regression $Y = w_1 X_3 + w_0 + \epsilon$.

# Basisfunktionserweiterung

Wir haben die Funktion $\phi(X_1, X_2) := X_1 X_2$ auf die ursprünglichen Daten angewendet.
Das ist eine Basisfunktionserweiterung mit $\phi$.

# Basisfunktionserweiterung

Wir haben die Funktion $\phi(X_1, X_2) := X_1 X_2$ auf die ursprünglichen Daten angewendet.
Das ist eine Basisfunktionserweiterung mit $\phi$.

**Definition**

Die lineare Regression mit Regressor $\phi(X_1, \ldots, X_k)$ wird lineare Regression mit Regressor $X_1, \ldots, X_k$ und Basisfunktionserweiterung mit $\phi$ genannt.

# Noch mehr Beispiele für Basisfunktionserweiterungen

Heinrich Heine Universität Düsseldorf

Wir kennen bereits den Trick $\phi(X) = (1, X)$ bzw. $\phi(X_1, \ldots, X_k) = (1, X_1, \ldots, X_k)$. Eine lineare Regression mit dieser Basisfunktionserweiterung ist wie eine lineare Regression mit Bias-Term $w_0$.

# Polynomiale Regression

Heinrich Heine
Universität Düsseldorf

Erweiterung mit $\phi(\mathrm{X}) = (1, \mathrm{X}, \mathrm{X}^2)$ nennt man quadratische Regression, allgemeiner $\phi(\mathrm{X}) = (1, \mathrm{X}, \mathrm{X}^2, \ldots, \mathrm{X}^{\mathrm{d}})$ polynomiale Regression von Grad d.

# Polynomiale Regression

Heinrich Heine
Universität Düsseldorf

Erweiterung mit $\phi(\mathrm{X}) = (1, \mathrm{X}, \mathrm{X}^2)$ nennt man quadratische Regression, allgemeiner $\phi(\mathrm{X}) = (1, \mathrm{X}, \mathrm{X}^2, \ldots, \mathrm{X}^{\mathrm{d}})$ polynomiale Regression von Grad d.
Auch die polynomiale Regression ist eine lineare Regression, denn
Sie ist linear in den Gewichten, nicht in den Features.

# Ab jetzt ins Notebook

Im Notebook schauen wir uns Overfitten bei der polynomialen Regression und dann ein Anwendungsbeispiel mit dem Melbourne Housing Datensatz an, bei dem wir auch Scikit-Learn's Pipelines kennen lernen.
Im Notebook lernen wir noch ein weiteres zentrales Thema im maschinellen Lernen kennen, was wir bei der linearen Regression anwenden: Regularisierung.

# Ab jetzt ins Notebook

Im Notebook schauen wir uns Overfitten bei der polynomialen Regression und dann ein Anwendungsbeispiel mit dem Melbourne Housing Datensatz an, bei dem wir auch Scikit-Learn's Pipelines kennen lernen.
Im Notebook lernen wir noch ein weiteres zentrales Thema im maschinellen Lernen kennen, was wir bei der linearen Regression anwenden: Regularisierung.

Den Ansatz der Basisfunktionserweiterung kann man noch wilder treiben - nächstes Mal beschäftigen wir uns dann mit den Kernel-Methoden.



Heinrich Heine
Universität Düsseldorf

# Vielen Dank für Ihre Aufmerksamkeit!